

TP4 Java Yassine Hamrouni

Exercice 1 - Eclipse

4- Écrire une classe Main qui affiche Hello Eclipse.

```
public class Main {  
    public static void main(String[] args){  
        System.out.println("Hello");  
    }  
}
```

5-1 Que fait sysout + Ctrl + Space dans un main ?

C'est un raccourci pour faire un System.out.println().

5-2 Que fait toString + Ctrl + Space dans une classe ?

Cela me propose 2 possibilités:

Soit d'Override la méthode toString() de la classe ou de créer directement une méthode de classe nommé toString()

5-3 Définir un champs foo de type int, que fait get + Ctrl + Space, et set + Ctrl + Space .

Cela me propose de créer un getter nommée getFoo() qui me permet de récupérer mon champ foo. Par contre pour set, cela ne me propose pas un setter nommé

setFoo(int newFoo), ce que je trouve bizarre ?

5-4 Dans le menu Source, comment générer un constructeur initialisant le champ foo ?

Sur le menu Source, il faut cliquer sur "Generate constructor using field"

```
public Test(int foo) {  
    super();  
    this.foo = foo;  
}
```

5-5 Sélectionner le nom de la classe puis Alt + Shift + R, qu'obtient-on ? Même question avec le champ foo .

Permet de modifier le nom de classe sur tout le code et également le nom du fichier, le constructeur. Pour une variable, ca modifie également sur tout le code.

5-6 Écrire `a = 2 + 3 + 4`, puis sélectionner `2 + 3` puis `Alt + Shift + L`

Cela stocke dans une variable la zone sélectionner.

```
int i = 2 + 3;
int a = i + 4;
```

5-7 Écrire `new Integer(2)`, en gardant le curseur après `'`', appuyer sur `Ctrl + 1`, que se passe-t-il ?

Propose de créer une variable local et d'affecter à ma variable courant, la variable local.

```
Integer integer = new Integer(2);
this.foo = integer;
```

5-8 Déclarer une variable `s` de type `String` et cliquer sur `String` en maintenant la touche `Ctrl`. Que se passe-t-il ?

Cela me ramène vers le code source de la classe `String`.

5-9 Dans la méthode `toString()`, que fait un `Ctrl + Clic` sur `super.toString()` ?

Ca nous ramène vers le code source de `toString()`.

```
public String toString() {
    return getClass().getName() + "@" + Integer.toHexString(hashCode());
}
```

Ce code retourne une chaîne de caractères représentant l'objet. Il inclut le nom de la classe de l'objet suivi de son code de hachage en format hexadécimal, séparés par le symbole `"@"`. C'est souvent utilisé pour identifier l'objet lors du débogage.

5-10 Sélectionner le champs `foo`, puis `Ctrl + Shift + G`. Que se passe-t-il ?

Cela me montre le chemin dans l'arborescence du projet où est le champ sélectionné en précisant quel est le type de ce dernier.

5-11 À quoi sert `Ctrl + Shift + O` ?

Organise les imports de manière alphabétique et cohérente dans le code courant. Supprime les imports inutilisés et ajoute les imports manquants.

5-12 À quoi sert Ctrl + Shift + C ?

Commente une ligne ou une partie de code.

Exercice 2 - Library

1- Écrire une classe Library avec un champs books de type ArrayList ainsi qu'un constructeur sans paramètre initialisant le champ books.

Attention à déclarer les bons modificateurs pour le champ books.

```
public class Library {  
    private ArrayList<Book> books;  
  
    public Library() {  
        this.books = new ArrayList<Book>();  
    }  
}
```

2- Ajouter une méthode add qui permet d'ajouter des books (non null) à la liste de livres.

```
public void add(Book book) {  
    books.add(book);  
}
```

3- Écrire une méthode findByTitle qui permet de trouver un livre en fonction de son titre dans la bibliothèque. La méthode doit renvoyer null dans le cas où aucun livre n'a le bon titre.

```
public Book findByTitle(String title) {  
    for (var book : books) {  
        if (book.title().equals(title)) {  
            return book;  
        }  
    }  
    return null;  
}
```

4- Comment le compilateur compile-t-il une boucle foreach sur une collection ?

Lorsqu'une boucle foreach est compilée, le compilateur la transforme en un itérateur à l'aide de la méthode iterator() de la collection, puis utilise cet itérateur pour parcourir la collection.

javap -c Library.class:

```
public src.Book findByTitle(java.lang.String);
```

Code:

```

0: aload_0
1: getfield      #17                // Field books:Ljava/util/ArrayList;
4: invokevirtual #32                // Method java/util/ArrayList.iterator:()Ljava/ut
7: astore_3
8: goto          34
11: aload_3
12: invokeinterface #36, 1           // InterfaceMethod java/util/Iterator.next:()Ljava
17: checkcast     #42                // class src/Book
20: astore_2
21: aload_2
22: invokevirtual #44                // Method src/Book.title:()Ljava/lang/String;
25: aload_1
26: invokevirtual #48                // Method java/lang/String.equals:(Ljava/lang/Obj
29: ifeq          34
32: aload_2
33: areturn
34: aload_3
35: invokeinterface #53, 1           // InterfaceMethod java/util/Iterator.hasNext:()Z
40: ifne          11
43: aconst_null
44: areturn

```

5- Expliquer pourquoi la méthode `findByTitle` doit renvoyer `null` plutôt que de lever une exception.

Cela permet à l'utilisateur de gérer le cas où aucun livre n'est trouvé de manière plus souple et plus adaptée à la logique de son application.

6- Écrire une méthode `toString` permettant d'afficher les livres de la bibliothèque dans l'ordre d'insertion, un livre par ligne.

```

@Override
public String toString() {
    var str = new String();
    for(var b: books) {
        str += b+"\n";
    }
    return str;
}

```

Exercice 3 - Librairie 2 (le retour de la vengeance)

1-Quelle est la complexité de la méthode `findByTitle` de la classe `Library` ?

Dans le pire des cas, la complexité de cette méthode est $O(n)$.

2- Regarder la javadoc de la classe `java.util.HashMap` ainsi que celle de ses méthodes `put` et `get`.

Quelle est la structure de données algorithmique dont `java.util.HashMap` est une implantation ?

La classe `HashMap` utilise des paires clé-valeur pour stocker les éléments. Elle utilise une fonction de hachage pour calculer un index où chaque paire est stockée dans le tableau interne de la classe.

Sachant que l'on veut améliorer la performance de `findByTitle` comment peut on utiliser la classe `java.util.HashMap` pour cela ?

Quelle sera alors la complexité de `findByTitle` ?

3- Commenter entièrement le code de la classe `Library` (pour ne pas perdre votre travail) et recopier les signatures des méthodes commentées. Pour l'instant, laisser la méthode `toString` de côté. Modifier les champs afin d'utiliser une `java.util.HashMap` et implanter les méthodes le constructeur et les méthodes `add` et `findByTitle`.

```
public Book findByTitle(String title) {  
    return books.get(title);  
}
```

La complexité de `findByTitle` avec cette approche est de $O(1)$, c'est-à-dire qu'elle permet une recherche en temps constant.

4- Expliquer pourquoi, ici, on a préféré utiliser une classe pour représenter `Library` plutôt qu'un record.

On a préféré utiliser une classe pour représenter `Library` plutôt qu'un record pour permettre une modification dynamique de la collection de livres (ajout, recherche par titre, etc.) et pour encapsuler le comportement spécifique à la gestion d'une bibliothèque, ce qui n'est pas directement supporté par la structure immuable et principalement destinée à la modélisation de données simples qu'offre un record.

5- Pour l'implantation de la méthode `toString`, quelle méthode de `java.util.HashMap` doit-on utiliser pour obtenir l'ensemble des valeurs stockées ? Si vous ne savez pas, lisez la javadoc !

Écrire la méthode `toString`.

```
@Override
public String toString() {
    var sb = new StringBuilder();
    for(var book : books.values()) {
        sb.append(book.toString()).append("\n");
    }
    return sb.toString();
}
```

6- En fait, la méthode `toString` ne fait pas exactement ce qui est demandé, car elle ne permet pas d'afficher les éléments dans l'ordre d'insertion. Sachant qu'il existe une classe `LinkedHashMap`, comment peut-on résoudre ce problème ?

Avec `LinkedHashMap`, l'ordre dans lequel les livres ont été ajoutés à la bibliothèque sera préservé, et la méthode `toString` reflétera cet ordre, garantissant ainsi que les éléments sont affichés dans l'ordre d'insertion.

```
this.books = new LinkedHashMap<>();
```

7- On souhaite ajouter une méthode `removeAllBooksFromAuthor` qui prend un nom d'auteur en paramètre et supprime tous les livres de cet auteur de la bibliothèque.

Sachant qu'il existe une méthode `remove` dans la classe `java.util.LinkedHashMap`, écrire une implantation qui parcourt tous les livres avec une boucle `for each` et supprime ceux de l'auteur avec `remove`.

Pourquoi votre implantation lève-t-elle une exception dans l'exemple suivant ?

L'utilisation d'une boucle `for-each` pour supprimer des éléments d'une `LinkedHashMap` directement avec la méthode `remove` lors du parcours cause une `ConcurrentModificationException`. Cette exception survient car modifier la collection pendant qu'elle est parcourue est interdit, car cela peut compromettre l'intégrité de l'itérateur utilisé implicitement par la boucle `for-each`. Pour éviter cela, il faut d'abord collecter les clés des éléments à supprimer dans une liste séparée, puis parcourir cette liste pour supprimer les éléments de la collection originale, évitant ainsi toute modification concurrente.

8- En fait, il existe une méthode `remove` sur l'interface `Iterator` qui n'a pas ce problème, car le parcours et la suppression se font sur le même itérateur.

Implanter correctement la méthode `removeAllBooksFromAuthor`.

```
public void removeAllBooksFromAuthor(String author) {
    var titlesToRemove = new ArrayList<String>();

    for (var entry : books.entrySet()) {
        if (entry.getValue().author().equals(author)) {
            titlesToRemove.add(entry.getKey());
        }
    }

    for (var title : titlesToRemove) {
        books.remove(title);
    }
}
```

9- il existe une méthode `removeIf` sur `Collection` qui permet d'écrire la méthode `removeAllBooksFromAuthor` en une ligne !

```
public void removeAllBooksFromAuthor(String author) {
    books.values().removeIf(book -> book.author().equals(author));
}
```